Counting Reads with featureCounts

2026-04-17

Introduction

After read alignment, the next step in most RNA-seq pipelines is summarising aligned reads into per-gene (or per-feature) count matrices for differential-expression analysis. `featureCounts` (in `Rsubread`) and HTSeq-count are the two standard tools; both consume BAM files plus a GTF annotation and produce a gene-by-sample count matrix. The choices that matter — strand-specificity, multi-mapper handling, paired-end mode, and meta-feature aggregation — directly affect downstream conclusions and must match the experimental library prep.

Prerequisites

A working understanding of BAM alignment files, GTF/GFF annotation formats, and the difference between transcript-level and gene-level expression quantification.

Theory

Each aligned read is assigned to a feature (gene, exon, or other annotated region) it overlaps. Key configuration choices:

- **Strand-specificity**: `strandSpecific = 0` (unstranded), `= 1` (forward-stranded, dUTP), `= 2` (reverse-stranded, TruSeq SR). Wrong setting halves the assigned counts.
- **Multi-mapper handling**: usually counted once at the primary alignment; fractional counting (`fraction = TRUE`) splits credit across all alignments of a multi-mapper.
- **Paired-end**: count fragments rather than reads; both mates must align consistently.
- **Meta-feature**: aggregate exon-level counts into gene-level by `gene_id`; alternatively, count at the exon level for splicing analyses.

For transcript-level counts, pseudo-alignment tools (Salmon, Kallisto) skip the BAM-producing alignment step and provide isoform-level abundance estimates directly.

Assumptions

The GTF annotation matches the genome reference used for alignment; the strand-specificity setting matches the library prep; aligned BAMs are coordinate-sorted (or sortable by `featureCounts`).

R Implementation

```r
library(Rsubread)

# Build counts from BAM + GTF
# counts <- featureCounts(files = c("s1.bam", "s2.bam"),
#                         annot.ext = "annotation.gtf",
#                         isGTFAnnotationFile = TRUE,
#                         GTF.featureType = "exon",
#                         GTF.attrType = "gene_id",
#                         strandSpecific = 2,
```

```
#                           isPairedEnd = TRUE)

# counts$counts: gene x sample matrix
```

Output & Results

`featureCounts()` returns a list with `$counts` (gene-by-sample matrix), `$annotation` (per-gene metadata including length used for FPKM/TPM normalisation), and `$stat` (per-sample summary of assigned vs ambiguous vs unassigned reads).

Interpretation

A reporting sentence: "featureCounts (Rsubread v2.16) assigned 82 % of mapped reads to GENCODE v44 gene features (`strandSpecific = 2`, `isPairedEnd = TRUE`); 8 % were ambiguous due to multi-gene overlap and 10 % were unassigned to any feature. The high assignment rate confirms the strand setting matches the dUTP library prep." Always quote the assignment rate; sub-50 % rates signal serious problems (wrong strand, annotation mismatch).

Practical Tips

- Report `$stat` summaries alongside the count matrix; low assignment rates (below 60 % for typical RNA-seq) signal alignment, annotation, or strand-setting problems.
- Strand-specificity must match library protocol exactly; check with `infer_experiment.py` from RSeQC or visual inspection in IGV when uncertain.
- Use `primaryOnly = TRUE` for ChIP-seq and ATAC-seq, where multi-mappers are typically excluded.
- For transcript-level (isoform) counts, switch to Salmon, Kallisto, or RSEM; gene-level counting collapses isoform-resolved information.
- Bundle counts with metadata in a `SummarizedExperiment` object for downstream Bioconductor workflows (DESeq2, edgeR, limma all accept `SummarizedExperiment`).
- Include feature length in the output and use it for normalisation when reporting TPM/FPKM; raw counts alone do not control for gene length differences.

R Packages Used

`Rsubread::featureCounts()` for the canonical implementation; `GenomicAlignments::summarizeOverlaps()` for an alternative Bioconductor-style approach; `tximport` for importing transcript-level estimates from Salmon/Kallisto into `SummarizedExperiment`; `tximeta` for metadata-aware imports.

For Reviewers

What to look for in a paper using this method.

- **Common misapplications.**
- **Diagnostics that should be reported but often aren't.**
- **Red flags in tables and figures.**
- **What to verify.**
- **What an adequate Methods paragraph must contain.**

See also — labs in this chapter

- RNA-seq with DESeq2 / edgeR
- Enrichment analysis (GSEA / ORA)
- scRNA-seq with Seurat